DLOps Assignment 5

Mamta Chauhan (M25CSA018)

Resources and Links

- **WandB Q1 Baseline:**
  https://wandb.ai/m25csa018-iit-j/Assignment5_Q1_ViT_Baseline?nw=nwuserm25csa018
- **WandB Q1 LoRA Experiments:**
  https://wandb.ai/m25csa018-iit-j/Assignment5_Q1_ViT_LoRA?nw=nwuserm25csa018
- **WandB Q2 (Adversarial Attacks):**
  https://wandb.ai/m25csa018-iit-j/dlops-q2-adversarial?nw=nwuserm25csa018
- **GitHub Repository:**
  https://github.com/mamta54/MLOps-Mamta-M25CSA018/tree/Assignment5

1 Question 1: ViT-S Fine-tuning with LoRA on CIFAR-100

1.1 Methodology

We fine-tuned a pre-trained Vision Transformer (ViT-S) model on CIFAR-100 using LoRA (Low-Rank Adaptation).

- Baseline: Only classification head trained
- LoRA applied on attention layers (Q, K, V)
- Hyperparameters explored:
  - Rank: 2, 4, 8
  - Alpha: 2, 4, 8
  - Dropout: 0.1

1.2 LoRA Experiment: Rank=2, Alpha=2

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	4.3000	8.08%	3.6413	22.40%
2	3.1806	36.61%	2.8940	44.55%
3	2.6470	54.18%	2.5523	55.05%
4	2.3502	62.23%	2.3263	61.00%
5	2.1446	67.53%	2.1651	64.45%
6	1.9896	70.67%	2.0452	67.35%
7	1.8710	74.13%	1.9600	69.05%
8	1.7657	76.08%	1.8861	69.00%
9	1.6806	78.05%	1.8208	71.55%
10	1.6043	80.08%	1.7524	72.80%

1.3 LoRA Experiment: Rank=2, Alpha=4

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	4.2726	8.50%	3.5713	23.70%
2	3.1683	36.45%	2.8812	43.75%
3	2.6778	51.26%	2.5597	53.05%
4	2.3803	59.62%	2.3727	56.80%
5	2.1687	64.88%	2.1826	61.70%
6	2.0005	68.40%	2.0589	65.30%
7	1.8664	71.74%	1.9506	66.85%
8	1.7522	74.67%	1.8834	68.55%
9	1.6574	76.96%	1.7949	70.05%
10	1.5686	79.18%	1.7538	71.95%

1.4 LoRA Experiment: Rank=2, Alpha=8

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	3.9437	15.71%	3.1144	35.35%
2	2.6988	47.81%	2.4728	54.95%
3	2.2207	62.58%	2.1722	62.25%
4	1.9558	70.21%	1.9883	69.10%
5	1.7692	75.45%	1.8580	71.15%
6	1.6236	78.71%	1.7706	72.50%
7	1.5086	81.75%	1.6965	73.80%
8	1.4090	83.97%	1.6185	74.70%
9	1.3268	85.55%	1.5700	76.20%
10	1.2554	87.21%	1.5386	76.25%

1.5 LoRA Experiment: Rank=4, Alpha=8

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	3.9108	17.40%	2.9735	39.10%
2	2.5312	51.25%	2.2547	60.50%
3	1.9886	66.50%	1.9000	66.85%
4	1.6665	74.45%	1.6827	71.90%
5	1.4418	79.09%	1.5092	75.55%
6	1.2672	83.16%	1.3891	77.85%
7	1.1254	85.74%	1.2940	79.35%
8	1.0028	88.12%	1.2255	78.90%
9	0.8977	89.84%	1.1928	80.05%
10	0.8135	91.61%	1.1400	79.95%

1.6 LoRA Experiment: Rank=8, Alpha=8 (Best)

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	3.8686	18.61%	2.9020	43.90%
2	2.4268	56.43%	2.1489	61.00%
3	1.8256	71.58%	1.7225	72.45%
4	1.4744	78.93%	1.4843	76.80%
5	1.2303	83.00%	1.3183	77.55%
6	1.0410	86.75%	1.1890	79.90%
7	0.8909	89.20%	1.1006	80.25%
8	0.7684	91.22%	1.0212	81.25%
9	0.6675	92.79%	0.9664	82.35%
10	0.5792	94.41%	0.9342	81.80%

1.7 Final Comparison

Configuration	Best Val Accuracy	Trainable Params	Trainable %
Rank=2, Alpha=2	72.80%	110K	0.128%
Rank=2, Alpha=4	71.95%	110K	0.128%
Rank=2, Alpha=8	76.25%	110K	0.128%
Rank=4, Alpha=8	80.05%	221K	0.256%
Rank=8, Alpha=8	**82.35%**	442K	0.512%

1.8 Observations

- Increasing rank and alpha improves performance significantly
- Rank=8, Alpha=8 achieves the best validation accuracy (82.35%)
- LoRA remains highly parameter-efficient ($< 1\%$ trainable parameters)
- Training shows stable convergence across all configurations

2 Part (i): FGSM Attack — From Scratch vs IBM ART

2.1 Dataset and Model

Dataset: CIFAR-10 — 60,000 color images (32×32) across 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). 50,000 training and 10,000 test images.

Model: ResNet18 trained from scratch (no pretrained weights). The standard ImageNet ResNet18 was adapted for CIFAR-10 by:

- Replacing the 7×7 stride-2 convolution with a 3×3 stride-1 convolution to preserve spatial resolution on 32×32 inputs.
- Removing the MaxPool layer to avoid aggressive downsampling.
- Replacing the final fully connected layer with a 10-class output head.

Training Configuration:

Hyperparameter	**Value**
Optimizer	SGD with Nesterov momentum
Learning Rate	0.1 (CosineAnnealingLR over 50 epochs)
Momentum	0.9
Weight Decay	5×10^{-4}
Batch Size	128
Epochs	50
Augmentation	RandomCrop(32, pad=4), RandomHorizontalFlip

2.2 Classifier Results

Metric	**Value**
Best Validation Accuracy	**94.31%**
Final Training Accuracy	99.88%
Requirement	$\geq 72\%$

The model comfortably exceeds the 72% accuracy requirement.

2.3 FGSM Attack: From Scratch

The Fast Gradient Sign Method (FGSM) [**?**] generates adversarial examples by taking a single gradient step in the direction that maximizes the classification loss:

$$x_{\text{adv}} = \text{clip}\left(x + \varepsilon \cdot \text{sign}\left(\nabla_x \mathcal{L}(\theta, x, y)\right),\ 0,\ 1\right) \tag{1}$$

where x is the original image, y is the true label, $\mathcal{L}$ is the cross-entropy loss, θ are the model weights, and ε is the perturbation budget.

Implementation steps:

1. Enable gradient tracking on the input tensor (`requires_grad = True`)
2. Forward pass to compute the classification loss
3. Backward pass to compute $\nabla_x \mathcal{L}$
4. Apply the sign of the gradient scaled by ε
5. Clip pixel values to $[0, 1]$

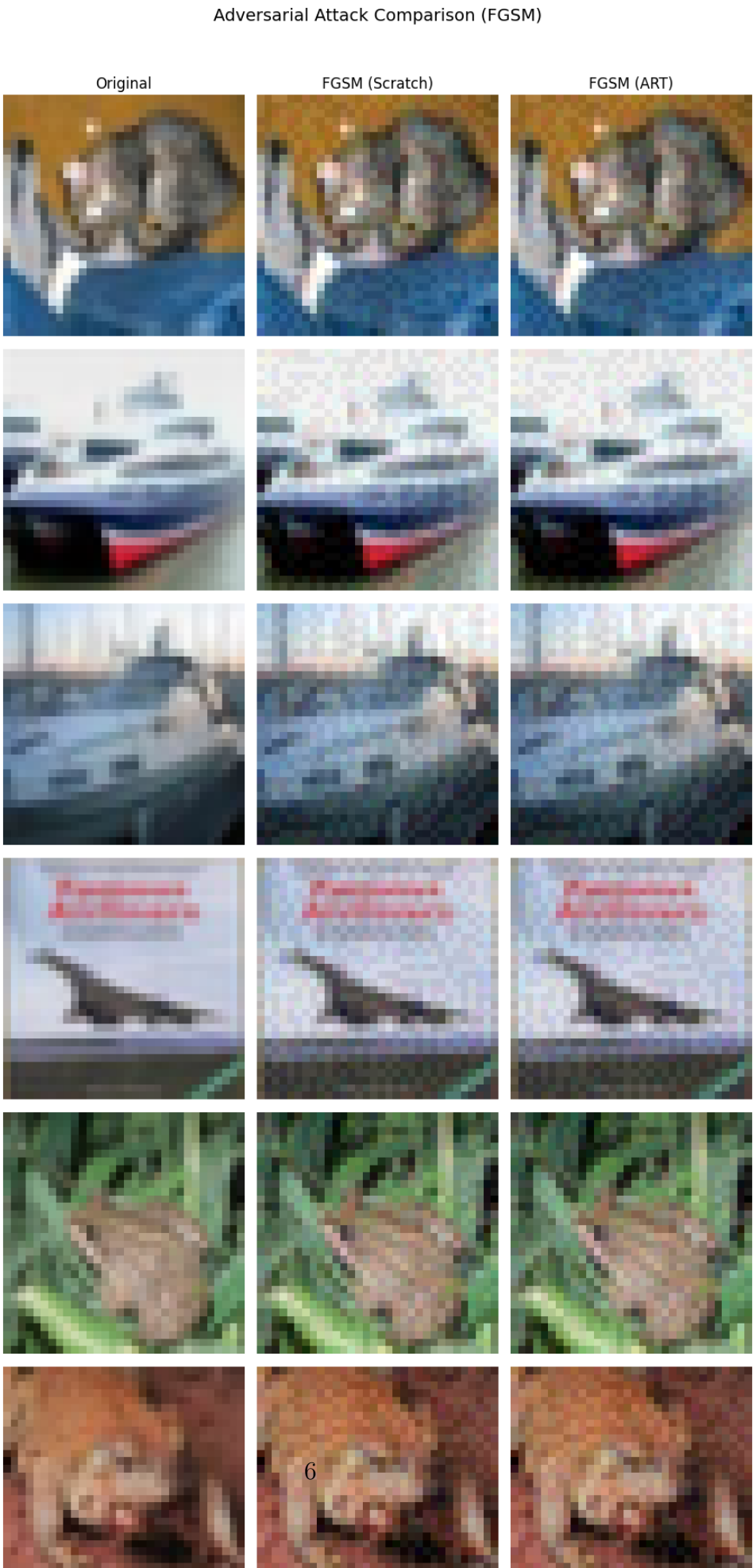
2.4 FGSM Attack: IBM ART

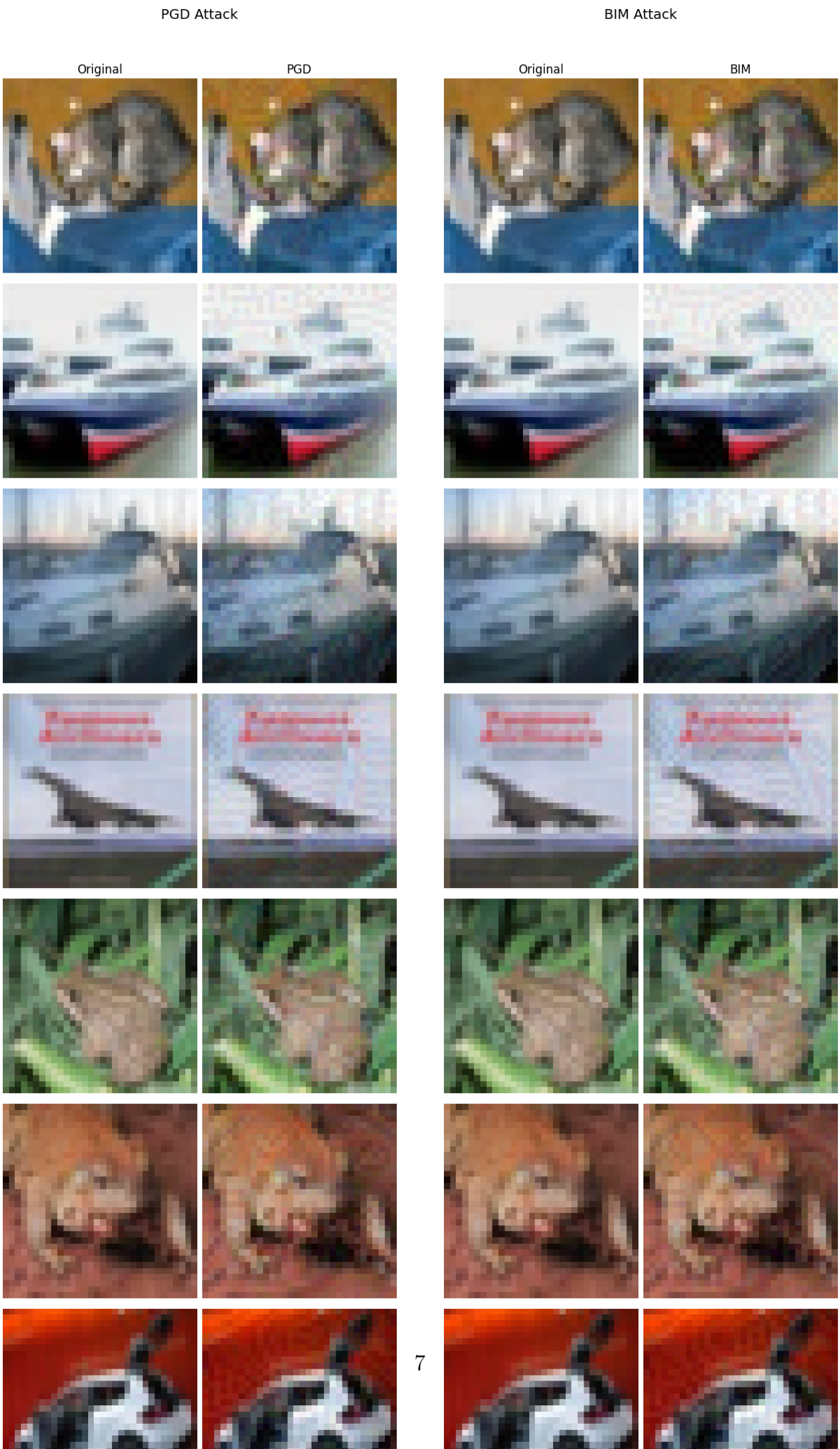
IBM ART's `FastGradientMethod` was used with a `PyTorchClassifier` wrapper. ART internally handles normalization via its preprocessing pipeline, so raw $[0, 1]$ images were passed directly.

2.5 Attack Results ($\varepsilon = 0.03$)

Attack	**Clean Acc**	**Adv Acc**	**Drop**
No Attack	94.31%	94.31%	0.00%
FGSM (Scratch)	94.31%	32.77%	61.54%
FGSM (ART)	94.31%	36.07%	58.24%
PGD (ART)	94.31%	3.45%	**90.86%**
BIM (ART)	94.31%	3.90%	**90.41%**

2.6 Visual Comparison




PGD Attack
BIM Attack
Original
PGD
Original
BIM

2.7 Perturbation Strength vs Accuracy Drop

The epsilon sweep evaluates FGSM (ART) at multiple perturbation budgets:

ε	Clean Acc	Adv Acc	Drop
0.010	94.31%	47.43%	46.88%
0.020	94.31%	39.90%	54.41%
0.030	94.31%	36.07%	58.24%
0.050	94.31%	28.65%	65.66%
0.100	94.31%	14.78%	79.53%

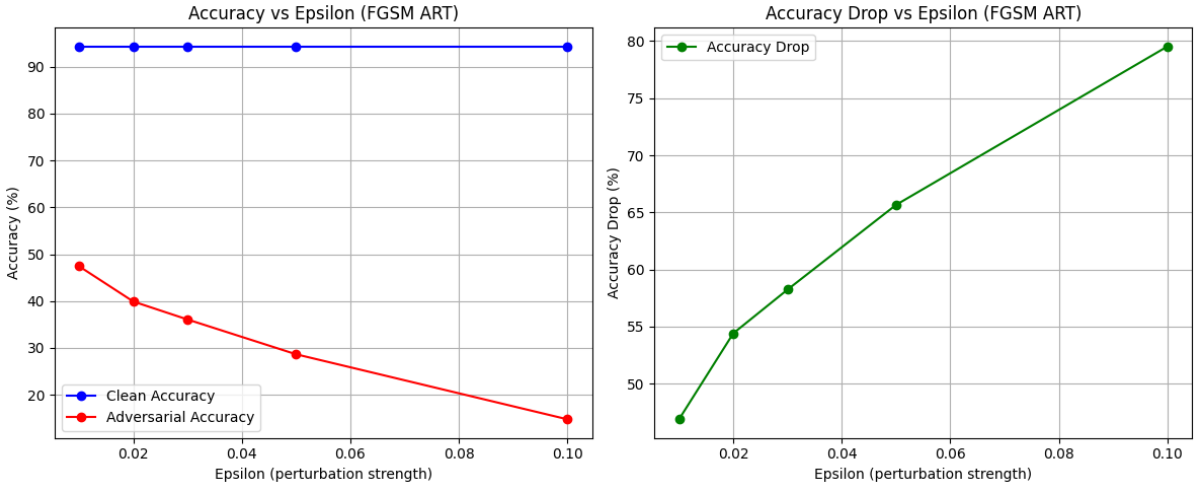


Figure 3: Accuracy vs epsilon (left) and accuracy drop vs epsilon (right). As ε increases, adversarial accuracy decreases monotonically.

2.8 Analysis

FGSM Scratch vs ART: Both methods produce similar accuracy drops (61.54% vs 58.24%), confirming the scratch implementation is correct. The small difference ($\approx 3\%$) is due to minor differences in normalization handling.

PGD vs BIM vs FGSM: PGD and BIM are substantially more powerful than FGSM, reducing accuracy to $\approx 3\%$ vs $\approx 33\%$ for FGSM. This is because PGD and BIM apply 40 iterative gradient steps, each one maximizing the loss further. PGD additionally uses a random start, making it marginally stronger than BIM.

Epsilon effect: Accuracy drops monotonically as ε increases. At $\varepsilon = 0.1$, accuracy falls to 14.78%, yet the perturbation becomes slightly visible to the human eye. At $\varepsilon = 0.01$, the perturbation is completely invisible but still causes a 46.88% accuracy drop — demonstrating the vulnerability of deep neural networks to imperceptible input modifications.

3 Part (ii): Adversarial Detection

3.1 Task Overview

Two binary classifiers (detectors) were trained using ResNet34 to distinguish between clean and adversarial images:

- **Detector A:** Trained on clean images + PGD adversarial images
- **Detector B:** Trained on clean images + BIM adversarial images

Labels: 0 = clean, 1 = adversarial.

3.2 Adversarial Image Generation

PGD (Projected Gradient Descent):

$$x^{t+1} = \text{Clip}_{\varepsilon}\left(x^t + \alpha \cdot \text{sign}\left(\nabla_x \mathcal{L}(\theta, x^t, y)\right)\right) \tag{2}$$

Parameters: $\varepsilon = 0.03$, $\alpha = 0.007$, iterations = 40, random initialization enabled.

BIM (Basic Iterative Method): Same as PGD but without random initialization — deterministic iterative FGSM.

3.3 Detector Architecture

ResNet34 adapted for CIFAR-10 (32×32) with the same modifications as ResNet18: 3×3 stride-1 first convolution, no MaxPool, and a 2-class output head.

Training Configuration:

Hyperparameter	Value
Optimizer	Adam (lr=10^{-3}, weight decay=10^{-4})
Scheduler	StepLR (step=5, $\gamma = 0.5$)
Epochs	15
Batch Size	128
Dataset	10,000 clean + 10,000 adversarial (80/20 split)

3.4 Detector Results

Detector	Overall Acc	Clean Det	Adv Det	Status
PGD Detector	**99.97%**	100.00%	99.95%	PASS
BIM Detector	**99.97%**	99.95%	100.00%	PASS
Requirement	$\geq 70\%$	—	—	

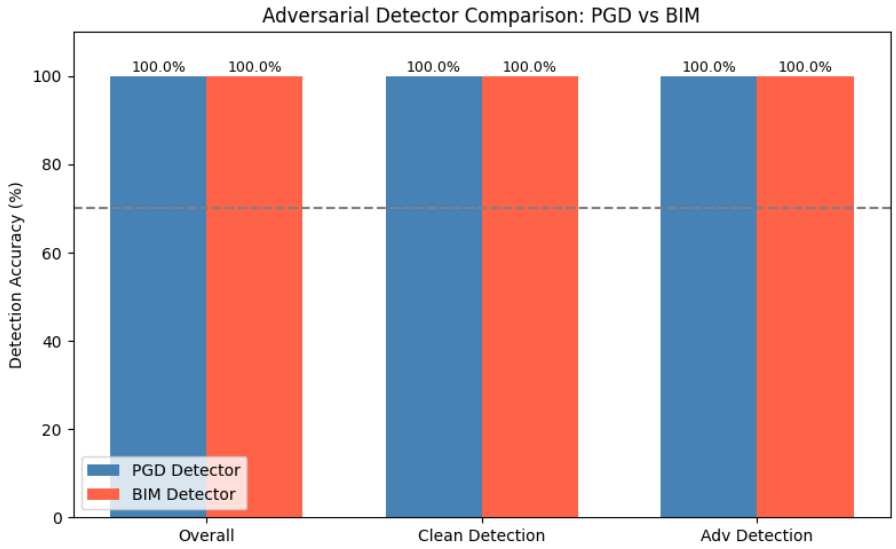


Figure 4: PGD vs BIM detector comparison. Both achieve near-perfect detection accuracy, well above the 70% requirement.

3.5 Analysis

Both detectors achieved $\approx 99.97\%$ overall accuracy, far exceeding the 70% requirement. This indicates that PGD and BIM adversarial images, despite being imperceptible to humans, leave a strong statistical signature that a classifier can learn to detect.

PGD vs BIM detection: The detection accuracy is nearly identical for both attacks (99.97% each). This is expected because BIM is a special case of PGD (without random initialization), so the adversarial perturbation patterns they produce are structurally similar, making them equally detectable.

Why detection is so accurate: Adversarial perturbations generated by gradient-based attacks tend to add structured high-frequency noise patterns that differ systematically from natural image noise. The ResNet34 detector learns to recognize these patterns. The near-perfect score also suggests the detector may be partially learning attack-specific artifacts rather than general adversarial patterns — cross-attack generalization would require training on a mixture of attack types.